IcePredict 결선 발표 대본 — 10 분 (발표 약 5 분 + 시연 약 5 분)

준비: 랜선(파이 ↔ 데스크톱 위쪽 포트) → 파이 터미널에서 sw/scripts/demo/venue_pi.sh (화면에 보이게) → 데스크톱 venue_desktop.sh (대기 화면, 발표 내내 켜 둔다: 1 노면 판정 · 2 1 차 주행 · 3 2 차 방어). 아이패드 사진은 전체 화면, 밝기 최대. RealSense 창 '재촬영 보정'은 끔.

쪽	시간	시연 동작	말할 것
1. 표지	0:00-0:20	—	IcePredict 입니다. 블랙아이스를 카메라로 미리 보고, 놓치면 노면을 직접 재서 제때 세우는 운전자 보조 장치입니다.
2. 문제	0:20-1:10	—	블랙아이스는 투명해서 카메라가 놓치는 경우가 반드시 남습니다. 실사진 얼음의 3.4 % 는 어떤 문턱으로도 못 잡았습니다. 그리고 지금 차의 긴급제동·간격 유지는 마찰을 모르면 마른 노면 기준으로 제동 시점을 잡습니다. 60 km/h 에서 마른 노면은 38 m 면 서지만 블랙아이스는 105 m 가 필요합니다.
3. 해법	1:10-1:50	—	그래서 두 겹으로 막습니다. 1 차는 카메라로 미리 보고, 2 차는 놓쳤을 때 노면을 재서 제동 시점을 고치고, 그 판단은 메인 컴퓨터와 분리된 안전 전용 RTOS 에서 돕니다. 조향은 운전자 몫이고, 시스템은 경고·가속 제한·제동만 돕습니다.
4. 1 차 방어	1:50-3:10	**시연 ① 데스크톱 1 키(노면 판정): 아이패드로 마른·젖은·블랙아이스 사진을 차례로 비춰 확률이 맞게 나오는 걸 보여 준다 → 2 키(1 차 주행): 화면 아래 '빙판까지 ○ m'가 150 m 이상일 때 블랙아이스 사진(대각선 줄무늬) → 빙판 앞 정지. 시간이 되면 2 키로 한 판 더, 이번엔 마른 노면 사진.**	실제 도로 사진 69,358 장을 STM32N6 보드에 넣어 쟀을 때 블랙아이스 96.6 % 를 경보했고 마른 노면 오경보는 0.3 % 입니다. 보드는 지금 다른 대회(F1TENTH, 일본)에 나가 있어 오늘 시연은 같은 모델을 라즈베리파이에서 돌립니다. 차가 80 km/h 로 달립니다 — RealSense 에 빙판 사진을 비추면 경고가 뜨고 빙판 앞에 섭니다. 마른 노면 사진이면 경고 없이 그대로 빙판에 들어가 미끄러집니다. 제동 여부는 RealSense 판정 하나로만 정합니다.
5. 2 차 방어	3:10-5:40	**시연 ② 데스크톱 3 키(2 차 방어) → D 로 OFF, 액셀 끝까지, 빨간 선 'BRAKE HERE'에서 브레이크(핸들은 따라 움직이기만). D 로 ON 해서 같은 동작.**	카메라가 놓치면 2 차 방어가 브레이크를 0.4 초 살짝 밟아 마찰을 잽니다. 빙판은 감속이 거기서 막히고 그 값이 마찰입니다. 그 마찰로 정지 거리를 다시 계산해 20 ms 마다 제동 시점을 정합니다. 먼저 말씀드립니다 — 같은 조건에서 비교하려고 회피 조향은 매번 똑같이 자동으로 넣었고, 브레이크는 제가 직접 밟습니다. 우리 시스템은 조향에 개입하지 않고 제동 시점만 바꿉니다. OFF: 빨간 선에서 밟고 피해도 빙판에서 미끄러져 충돌합니다. ON: 탭 → 진동·경고 → 미리 감속 → 같은 지점에서 밟고 피하면 섭니다.
6. 결과	5:40-6:30	—	같은 C 코드에 마찰 값만 다르게 넣은 CARLA 비교입니다. 마른 노면 기준은 9 회 중 2 회만 섰고, 마찰 인식은 8 회 섰으며 그중 7 회는 목표대로 앞차 4 m 앞이었습니다.
7. 안전 전용 RTOS	6:30-8:30	**시연 ③ CARLA 로 돌아가 3 키(또는 R) → 2 차 방어 ON(패널 '파이 메인 프로그램: 동작' 초록 확인) → 출발 → 빨간 배너 '빙판 감지 μ 0.21'이 뜬 뒤에 K (전에 누르면 무시된다) → 빨간 선에서 브레이크 → 정지. 파이 터미널에 '[pi_main] KILL 받음'. 다음 판은 패널이 다시 초록(약 20 s)이 된 뒤.**	메인 컴퓨터는 카메라 AI, 지도, 통신처럼 무거운 일을 동시에 합니다. 그래서 소프트웨어 버그, 과부하, 업데이트 오류, 발열 같은 이유로 언제든 멈출 수 있습니다. 실제로 부하를 걸었을 때 일반 리눅스 프로그램은 최악 206 ms 까지 늦어졌습니다. 그래서 자동차는 브레이크 같은 안전 기능을 메인 컴퓨터에 맡기지 않습니다. 우리도 2 차 방어 판단을 메인과 떼어 안전 전용 RTOS 에 두었습니다. 곡선 빙판에서 파이를 멈추면 2 차가 파이에 있을 때 53 km/h, 안전 전용 RTOS 에 있을 때 39 km/h 로 충돌했습니다. 지금 보여 드리겠습니다. 빙판을 감지해 마찰을 잰 직후, 메인 프로그램을 실제로 강제 종료합니다. 파이와의 연결이 끊기는 게 아니라, 파이 안에서 카메라와 지도를 맡던 메인 프로그램만 죽습니다. 따로 떼어 둔 2 차 방어는 심장박동이 끊긴 걸 0.1 초 안에 알아채고, 이미 잰 마찰만으로 끝까지 제동해 차를 세웁니다. 정직하게 말씀드리면 STM32 보드는 연구실의 다른 대회(F1TENTH 레이싱, 일본) 장비로 나가 있어 같은 C 코드를 파이 격리 코어에서 돌렸고, 리눅스 커널이 통째로 멈추면 같이 멈춥니다. 그래서 실차 설계는 별도의 안전 전용 칩(MCU)입니다.
8. 정리	8:30-10:00	—	미리 보고, 놓치면 재서, 죽어도 판단합니다. 남은 과제는 운전자가 못 느끼게 마찰을 재는 방법과 보드 실측입니다. 소스와 근거 문서는 저장소에 모두 있습니다. 감사합니다.

키: 1 노면 판정 · 2 1 차 주행 · 3 2 차 방어 · C 휠 영점 · D 2 차 방어 켜기/끄기 · R 다시 · K 파이 메인 프로그램 죽이기 · I 빙판 표시 · T 3 인칭 창 · (RealSense 창) +/− 판정 네모, B 재촬영 보정

비상: 시연이 안 되면 제출 영상(IcePredict_시연영상_최종.mp4)을 튼다. 2 차 방어가 '응답 없음'이면 랜선·venue_pi.sh 확인.